DocuMind - Sujet du projet

Cahier des charges autonome - D0 - 9 novembre 2026

1. Contexte

Une organisation fictive reçoit des factures, bons de commande, PDF, fichiers CSV et classeurs Excel. Les informations sont dispersées, les contrôles sont manuels et les tableaux de bord ne permettent pas toujours de remonter à la source d'un chiffre.

2. Mission

Construire DocuMind, plateforme locale d'intelligence documentaire. Elle doit réceptionner des fichiers autorisés, extraire des données utiles, vérifier leur cohérence, historiser les traitements, produire des indicateurs et répondre à des questions avec un assistant IA local citant ses sources.

Le projet démontrera la complémentarité de Django REST, Python/Pandas, PostgreSQL, Airflow, Metabase, ChromaDB, Ollama et LangGraph. Une interface React/TypeScript offrira un point d'entrée unique. Aucune dépendance AWS ou API IA payante.

3. Utilisateurs

- Opérateur : importe, consulte l'état et relance des traitements autorisés.
- Analyste : consulte les rapports et interroge les documents sans modifier la source.
- Administrateur : gère les accès, paramètres et journaux locaux.

4. Sources

- PDF textuels avec pages extractibles ;
- CSV UTF-8 structurés ;
- classeurs XLSX sans macros ;
- corpus de démonstration entièrement synthétique ou explicitement autorisé.

OCR, scans, pièces manuscrites et données confidentielles réelles sont hors périmètre V1.

5. Fonctionnalités attendues

- F01 Import : déposer un document ou un lot, contrôler type, taille et empreinte SHA-256, attribuer un ID et conserver la source dans un stockage non public.
- F02 Traçabilité : enregistrer auteur, date, fichier, versions, étapes, erreurs et événements d'audit.
- F03 États : RECEIVED -> VALIDATING -> ACCEPTED -> INDEXED ; REJECTED pour un échec avec motif et relance autorisée.

- F04 Extraction : extraire du PDF textuel et des tableaux CSV/XLSX, sans supposer que tous les documents possèdent les mêmes colonnes.
- F05 Normalisation : identifier fournisseur, facture, date, devise, HT, TVA et TTC selon des règles explicites ; conserver la page ou ligne source.
- F06 Contrôles : vérifier champs obligatoires, formats, doublons et cohérence HT + TVA = TTC ; mettre en quarantaine les anomalies.
- F07 API : CRUD autorisé de documents, imports, états, anomalies et agrégats via Django REST ; documentation OpenAPI.
- F08 Airflow : DAG de traitement et actualisation planifiée avec reprise idempotente et journal d'exécution.
- F09 BI : exposer à Metabase des vues PostgreSQL validées : volumes, anomalies, factures et dépenses par fournisseur/période.
- F10 RAG : indexer des extraits et métadonnées dans ChromaDB, rechercher les sources puis générer une explication avec Ollama et LangGraph.
- F11 Réponses étayées : citer document, page/ligne et passage ; indiquer l'absence de preuve lorsque nécessaire.
- F12 Démo : jeu de données reproductible et scénarios de succès, rejet, reprise, recherche et abstention.

6. Règles métier

- La source d'un montant métier est PostgreSQL après validation, pas une valeur devinée par le LLM.
- Aucun fichier rejeté ne contribue aux KPI.
- Le rejeu d'un même import/version ne crée pas de doublon métier.
- Toute donnée extraite reste liée à son document et à sa provenance.
- L'index vectoriel ne remplace pas la base relationnelle pour les agrégats.
- Les instructions présentes dans les documents importés sont des données non fiables, jamais des ordres au système.
- L'assistant ne peut ni lancer des commandes système, ni écrire librement dans PostgreSQL, ni envoyer des documents à une API distante.
- Toute action sensible exige une permission côté backend et une trace d'audit.

7. Architecture imposée

- Frontend : React + TypeScript.
- Backend : Python + Django REST Framework, OpenAPI.
- Métadonnées et référentiel : PostgreSQL.
- Extraction/qualité : Python et Pandas.
- Orchestration : Apache Airflow.
- Tableaux de bord : Metabase sur vues SQL en lecture seule.

- Recherche documentaire : ChromaDB.
- Assistant local : Ollama + LangGraph.
- Services reproductibles : Docker Compose, démarrage par profils.
- Tests : pytest, tests d'API, tests d'intégration et scénario de bout en bout.

8. Modèle de données minimal

Document (id, fichier, format, sha256, état, horodatages), ProcessingRun (id, document, version, état, erreur), ExtractedRecord (id, document, run, source_locator, champs), QualityIssue (règle, sévérité, diagnostic), Supplier, Invoice (date, HT, TVA, TTC, devise), ChunkReference (document, page, index) et AuditEvent (acteur, action, cible, résultat).

Les migrations et cardinalités seront précisées lors de D1.

9. Contraintes non fonctionnelles

- Exécution locale sous Windows 11/Docker, sans service cloud payant.
- Services séparables pour limiter l'utilisation mémoire ; l'API doit rester disponible sans LLM.
- Données de test fictives ; aucun secret, upload, extrait privé ni base réelle dans Git.
- Refus des extensions ou structures non autorisées, prévention des traversées de chemin, limitation des tailles, validation des autorisations.
- Logs et erreurs exploitables, tests automatisés, documentation claire et restauration du corpus synthétique.

10. Livrables

- Dépôt Git avec README, sources, Docker Compose, .env.example et documentation.
- Diagrammes UML (composants, classes, séquence et états).
- API OpenAPI, migrations SQL et tests.
- Pipelines Python/Pandas et DAG Airflow.
- Vues analytiques et dashboard Metabase.
- Assistant local RAG/Ollama/LangGraph sourcé.
- Corpus synthétique, recette, captures authentiques et démonstration reproductible.

11. Critères de validation

- V01 : importer les trois formats de référence et obtenir un état et une trace par document.
- V02 : rejouer le traitement sans doubler les factures ni les KPI.
- V03 : rejeter un document invalide, enregistrer sa cause et l'exclure des agrégats.
- V04 : vérifier la concordance des métriques Metabase avec des requêtes SQL indépendantes.
- V05 : répondre à une question présente avec citation vérifiable ; s'abstenir sur une question absente.

- V06 : vérifier permissions, validations de fichiers et non-exfiltration.
- V07 : exécuter tests API/Python et scénario complet, avec preuves et limites consignées.

12. Hors périmètre V1

OCR des scans, documents manuscrits, comptabilité réelle, paiements, ERP, décisions financières automatiques, LLM entraîné sur mesure, intégrations cloud et hébergement public d'une API.

13. Plan

D0 : cahier des charges, architecture, UML, critères. D1 : API et PostgreSQL. D2 : Pandas et Airflow. D3 : Metabase. D4 : RAG et IA locale. D5 : stabilisation et V1.0.

État au D0 : spécifications uniquement ; aucune fonctionnalité logicielle validée à ce jour.